

Part 2 Report

Students: Ayman abu Hijleh(12112985), fadi alhindi(12112857)

1. Introduction

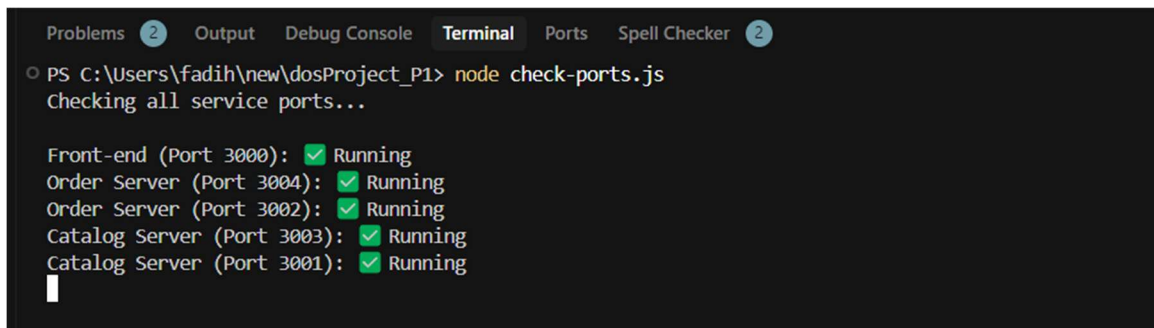
This part builds upon the Bazar.com project developed in part 1 by introducing three key enhancements: replication, caching, and cache consistency. The goal is to improve the system's scalability and performance under increased workload. The new architecture incorporates in-memory caching for faster query responses, server replication with load balancing for high availability, and cache invalidation mechanisms to ensure data consistency.

2. System Architecture and Design

In this part, the system includes the following components:

- Front-End Server: Handles all client requests and performs caching, load balancing, and request routing.
- Catalog Server Replicas: Two replicas responsible for handling queries and updates related to book metadata.
- Order Server Replicas: Two replicas handling book purchases and updating inventory.
- In-Memory Cache: Stores recent query results at the front-end to reduce response times for frequently accessed data.

-> System Initialization – all ports run:



```
Problems 2 Output Debug Console Terminal Ports Spell Checker 2
PS C:\Users\fadih\new\dosProject_P1> node check-ports.js
Checking all service ports...

Front-end (Port 3000): ✓ Running
Order Server (Port 3004): ✓ Running
Order Server (Port 3002): ✓ Running
Catalog Server (Port 3003): ✓ Running
Catalog Server (Port 3001): ✓ Running
```

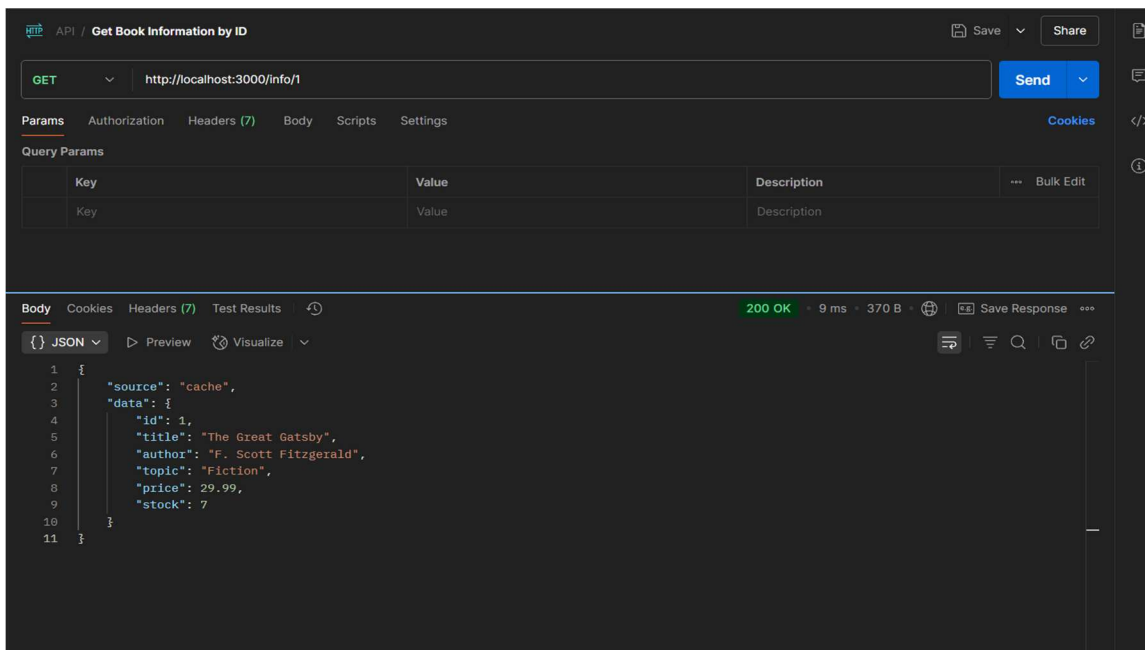
->Load balancing round robin:

```
Front-end server started on port 3000
###-----###
Map(0) {}
Selected catalog server: http://localhost:3001
fiction
  is deleted
###-----###
Map(0) {}
Selected catalog server: http://localhost:3003
###-----###
```

3. Caching Mechanism

The front-end maintains an in-memory cache to serve repeated read requests quickly. Cached responses significantly reduce latency for queries that access the same book information multiple times.

-> First GET request from cache (9ms) – first get book 9ms from cache:



->Second search from cache (4ms) – second search book 4ms from cache:

API / Search Books by Topic

GET Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Headers 7 hidden

Key	Value	Description
Key	Value	Description

Body Cookies Headers (7) Test Results

200 OK · 4 ms · 585 B Save Response

JSON Preview Visualize

```
1 {
2   "source": "cache",
3   "data": [
4     {
5       "id": 1,
6       "title": "The Great Gatsby",
7       "author": "F. Scott Fitzgerald",
8       "topic": "Fiction",
9       "price": 29.99,
10      "stock": 7
11    },
12    {
13      "id": 2,
14      "title": "To Kill a Mockingbird",
15      "author": "Harper Lee",
16      "topic": "Fiction",
17      "price": 8.99,
18      "stock": 2
19    }
20  ]
21 }
```

->First GET request from server (13ms) – first get book 13ms from server:

API / Get Book Information by ID

GET Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (7) Test Results

200 OK · 13 ms · 386 B Save Response

JSON Preview Visualize

```
1 {
2   "source": "http://localhost:3000",
3   "data": {
4     "id": 1,
5     "title": "The Great Gatsby",
6     "author": "F. Scott Fitzgerald",
7     "topic": "Fiction",
8     "price": 29.99,
9     "stock": 7
10   }
11 }
```

-> Search request from server (40ms) – first search book 40ms from server:

The screenshot shows a REST client interface with the following details:

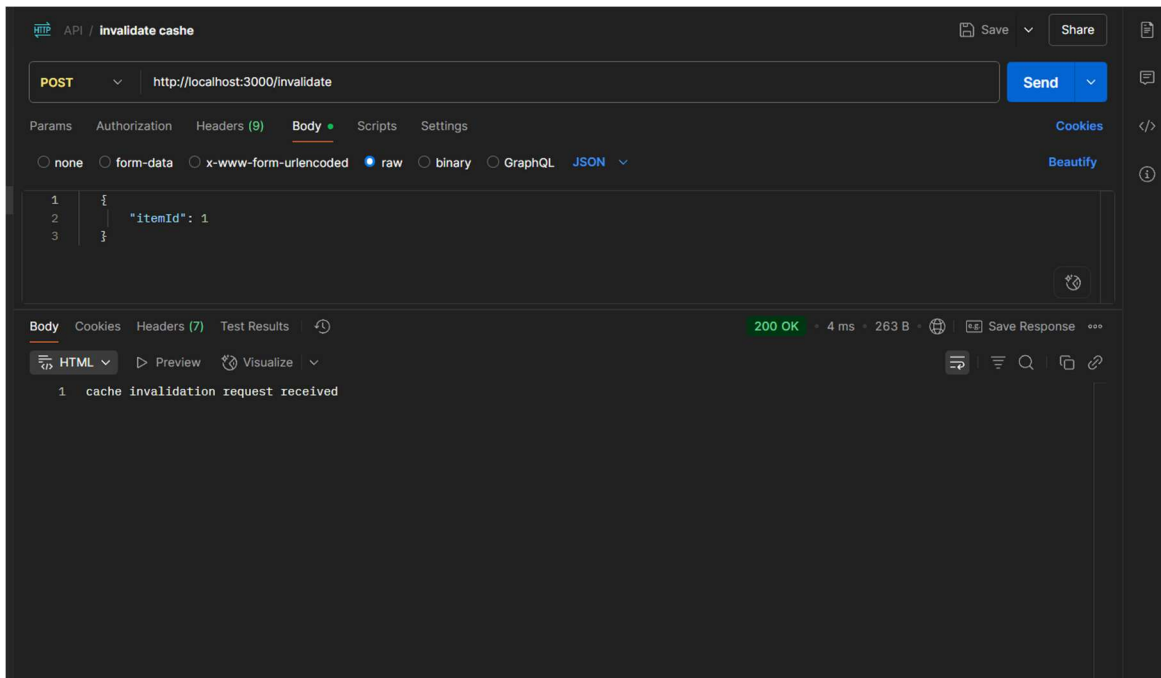
- Method:** GET
- URL:** http://localhost:3000/search/fiction
- Status:** 200 OK
- Response Time:** 40 ms
- Response Size:** 601 B
- Headers (7):** 7 hidden
- Body (JSON):**

```
1 {
2   "source": "http://localhost:3001",
3   "data": [
4     {
5       "id": 1,
6       "title": "The Great Gatsby",
7       "author": "F. Scott Fitzgerald",
8       "topic": "Fiction",
9       "price": 29.99,
10      "stock": 7
11    },
12    {
13      "id": 2,
14      "title": "To Kill a Mockingbird",
15      "author": "Harper Lee",
16      "topic": "Fiction",
17      "price": 8.99,
18      "stock": 2
19    }
20  ]
21 }
```

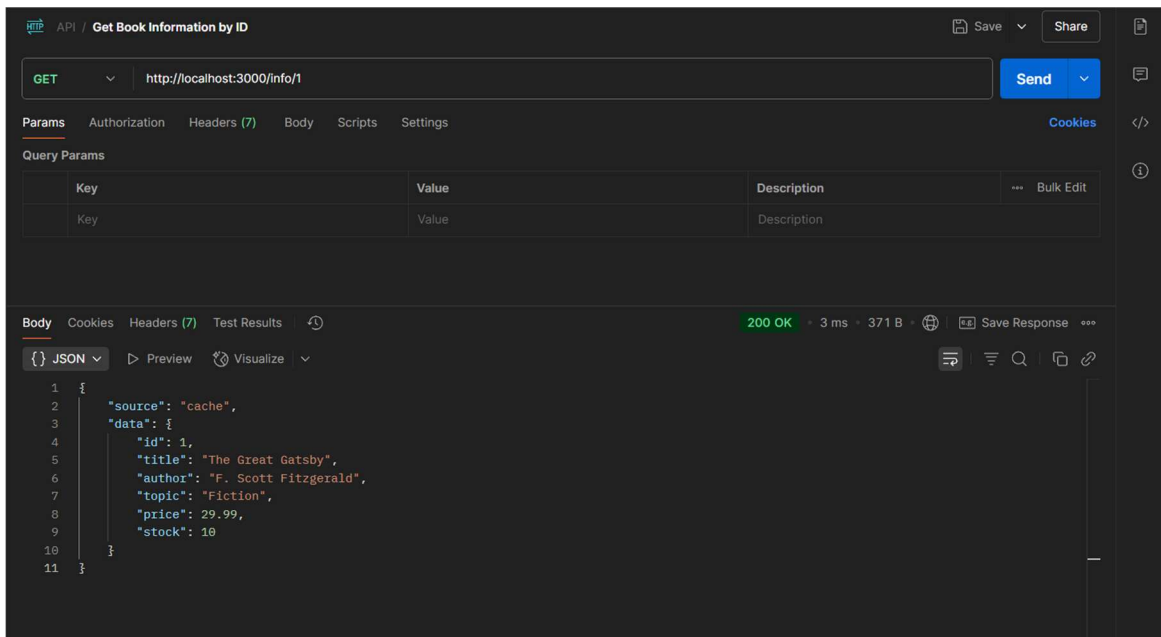
4. Cache Consistency

To maintain strong consistency, the system implements a cache invalidation protocol. When a write operation is performed (such as purchasing a book or updating inventory), the responsible replica sends an invalidate message to the cache before making the change.

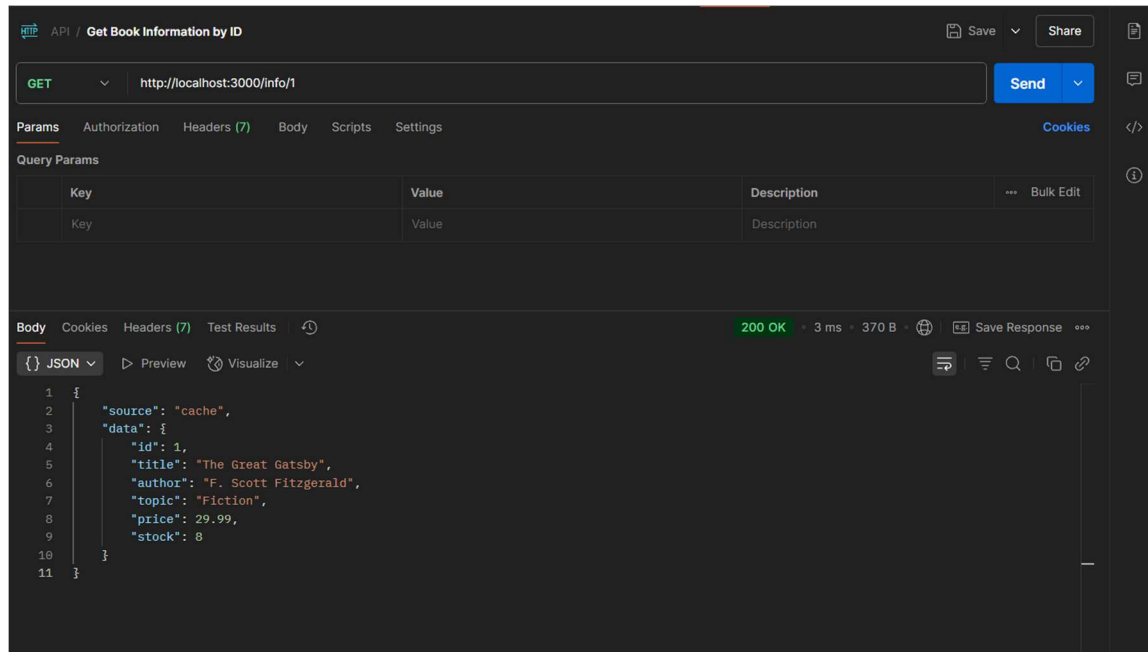
->Cache invalidated before update – invalidate cache:



->After cache invalidation, fresh data is fetched – after invalidta chashe:



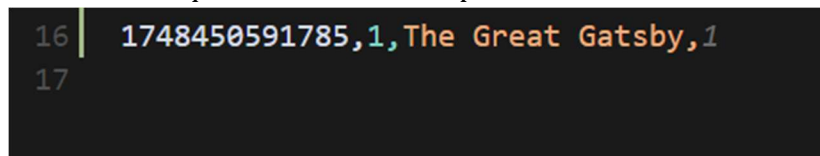
->Inconsistent data returned from stale cache – before make invalidta cache it return wrong stocks from cache:



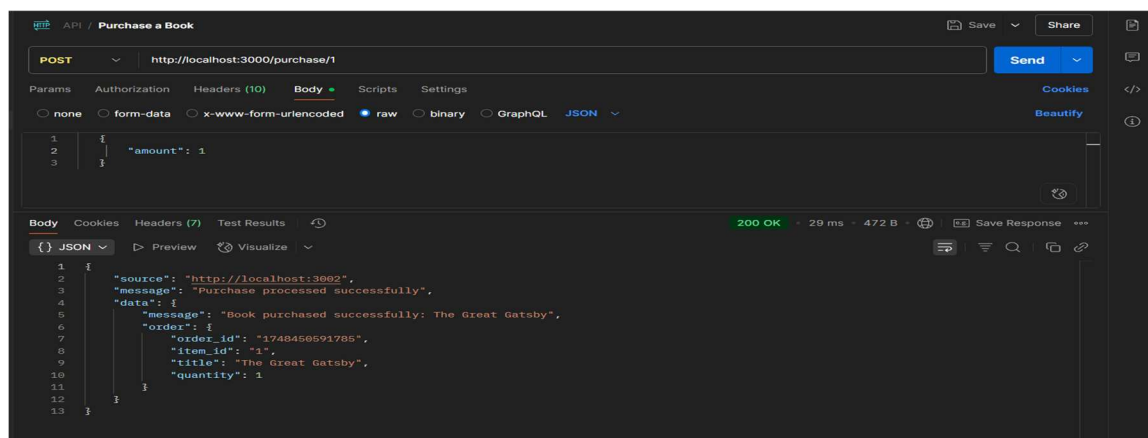
5. Order Processing and Inventory Updates

When a user purchases a book, the front-end routes the request to one of the order server replicas. The server checks availability and updates the stock accordingly. Cache is invalidated before stock updates to avoid serving outdated information.

->Purchase request sent – order for purchase :



-> Successful book purchase – purchase book:



->Database update (CSV) reflecting the order – proj cvs -1 for purchase book:

```
id,title,author,topic,price,stock
1,The Great Gatsby,F. Scott Fitzgerald,Fiction,29.99,9
2,To Kill a Mockingbird,Harper Lee,Fiction,8.99,2
3,1984,George Orwell,Dystopian,14.99,0
4,The Catcher in the Rye,J.D. Salinger,Fiction,9.99,3
5,The Alchemist,Paulo Coelho,Adventure,12.99,4
6,A Brief History of Time,Stephen Hawking,Science,15.99,1
7,The Art of War,Sun Tzu,Philosophy,6.99,10
8,The Diary of a Young Girl,Anne Frank,History,7.99,0
9,Brave New World,Aldous Huxley,Dystopian,13.99,6
10,Moby Dick,Herman Melville,Adventure,11.99,2
```

->Inventory updated for a book – update stock for book :

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/update`. The request body is a JSON object: `{ "id": 1, "stock": 10, "price": 29.99 }`. The response is a 200 OK status with a JSON body: `{ "source": "http://localhost:3001", "message": "Update processed successfully", "data": { "id": 1, "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "topic": "Fiction", "price": 29.99, "stock": 10 } }`.

```
PUT http://localhost:3000/update

{
  "id": 1,
  "stock": 10,
  "price": 29.99
}
```

200 OK · 18 ms · 429 B

```
{
  "source": "http://localhost:3001",
  "message": "Update processed successfully",
  "data": {
    "id": 1,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "topic": "Fiction",
    "price": 29.99,
    "stock": 10
  }
}
```

6. Experimental Results

We measured query response times with and without caching:

Query Type	Response Time	Source
First info request	13ms	Server
Cached info request	9ms	Cache
Cached search	4ms	Cache
First search	40ms	Server

These results confirm that caching provides significant performance improvement, particularly for repetitive queries. Additionally, we observed the importance of cache invalidation. Without it, users might receive outdated stock information, leading to inconsistencies.

7. Discussion and Improvements

The system shows clear gains in response time using caching and can handle higher loads thanks to replication and load balancing. However, improvements could include:

- Implementing Least Recently Used (LRU) cache eviction policy.
- Adding write-through cache to reduce invalidation overhead.
- Monitoring and dynamically adjusting load balancing based on replica health/performance.

8. Conclusion

This lab successfully demonstrates how caching, replication, and consistency mechanisms can greatly enhance the performance and reliability of distributed systems. Through careful cache management and load distribution, Bazar.com can now serve more users with better latency and consistent data.